
Project Report: Smartphone Price Prediction

Prepared for:

Dr. Fatma ElShehaby

Data Science and Methodology
(CSE221)

Prepared by:

Abdelrahman El-Khashab 23-101182

Abdullah Helmy 23-101239

Mariam Hassan 23-101309

Maryam Rageh 23-101297

Mohamed Elshabouri 23-101334

Academic Year 2026

Table of Contents

1. Abstract.....	5
2. Introduction.....	6
Background and Context:	6
Literature Review:	6
Project Objectives:.....	6
3. Materials and Methods.....	8
3.1 Data Description.....	8
3.2 Data Cleaning.....	8
3.3 Methodology	8
3.3.1 Scaling and normalization:.....	8
3.3.2 Regression models:.....	9
3.3.3 Classification models:.....	10
3.3.4 Data Visualization Techniques:	11
3.4 Model Evaluation Metrics.....	12
3.4.1 K-Fold Cross-Validation:	12
3.4.2 Coefficient of Determination (R^2 Score):	13
3.4.3 Classification Accuracy:	13
4. Results.....	14
4.1 Regression Findings and Performance.....	14
4.1.1 Linear Regression.....	14
4.1.2 Ridge Regression.....	15
4.1.4 Poly-Ridge Regression.....	15
4.1.5 Random Forest Regression.....	16
4.1.6 Regression – Model Performance.....	16
4.2 Classification Findings and Performance.....	18
4.2.1 Logistic Regression.....	18
4.2.2 Random Forest Classification	20
4.2.3 Naïve Bayes Classification.....	22

4.2.4 K-Nearest Neighbors Classification	28
4.2.5 Pricing Category Classification – Model Performance.....	31
4.2.6 Rating Category Classification – Model Performance.....	32
5. Conclusions.....	34
5.1 Summary of findings	34
5.1.1 Regression.....	34
5.1.2 Classification	34
5.2 Recommendations.....	35
5.3 Limitations	36
5.4 Future Work.....	36
6. Acknowledgements.....	37
Team Contributions.....	37
7. References.....	38

TABLE OF FIGURES

Figure 1 (Visualizing the Target Log Transformation)	9
Figure 2 (Actual vs. Predicted Price: Best vs. Worst Folds)	12
Figure 3 (Actual vs. Predicted Price: Linear Regression).....	14
Figure 4 (Cumulative Average Test R2 Across Folds)	15
Figure 5 (Pixels and GHz vs. Price)	16
Figure 6 (Actual vs. Predicted All Models)	17
Figure 7 (Model Performance Comparison: Train vs. Test R2)	18
Figure 8 (Logistic Regression: Where the Linear Model Fails)	19
Figure 9 (Overlapping Feature Space: RAM_GB vs. RATING).....	20
Figure 10 (Confusion Matrix - Random Forest)	21
Figure 11 (Distribution of Actual vs. Predicted Price Categories)	22
Figure 12 (dual perspective on the Naïve Bayes classification task).....	23
Figure 13 (Confusion Matrix - Naive Bayes Rating).....	25
Figure 14 (Distribution of Smartphone Ratings).....	26
Figure 15 (Box Plot of Phone Ratings)	27
Figure 16 (Confusion Matrix - Rating Prediction)	28
Figure 17 (KNN Accuracy for Different K Values)	29
Figure 18 (Confusion Matrix - K-Nearest Neighbors Model)	31

Figure 19 (Smartphone Pricing Classification: Model Performance Comparison)	32
Figure 20 (Model Performance Comparison: Naive Bayes vs KNN)	33
Table 1 (Random Forest Classification Model Performance)	21
Table 2 (NAIVE BAYES PRICE BRACKET EVALUATION)	23
Table 3 (Gaussian Naive Bayes Model Performance)	24
Table 4 (K-Nearest Neighbors Model Performance)	30
Table 5 (Contribution Table)	37

1. Abstract

This project aims to predict smartphone prices and classify devices into meaningful categories using machine learning techniques (Regression and Classification) applied to a dataset of smartphone specifications. The core problem addressed was understanding which hardware features most significantly drive smartphone pricing, and whether price and quality tiers could be reliably predicted from technical specs alone.

The analysis revealed strong correlations between price and features such as RAM, clock speed, and total camera megapixels. After thorough data cleaning, including handling missing values, encoding categorical variables, and log-transforming the skewed price distribution, five regression models were trained and evaluated. The Random Forest Regression Model emerged as the top performer, achieving the highest R^2 score on the test set, outperforming Linear, Ridge, and Polynomial variants. For classification, the Random Forest Classifier also took the lead in predicting price categories (Budget / Mid / Premium), while KNN performed competitively in classifying user rating tiers.

The main recommendation is to prioritize ensemble-based models for real-world pricing predictions, given their robustness to non-linear feature relationships. Future work could incorporate brand reputation, release year, and regional pricing data to further improve predictive accuracy and generalizability across global markets.

2. Introduction

Background and Context:

The global smartphone market releases hundreds of devices annually across a wide range of price points, making it difficult for consumers to assess whether a device is fairly priced relative to its specifications.

Smartphone pricing is driven by a combination of quantifiable hardware — processing power, memory, camera capability, and connectivity — and softer forces such as brand positioning. This makes it a suitable problem for machine learning: there is enough structured signal in device specifications to train predictive models, while the relationships between features and price are too complex for manual rule-based approaches.

Literature Review:

Smartphone price prediction has been an active area of applied machine learning research, framing the problem as both regression and classification — the same dual formulation adopted in this project.

Studies have compared a wide range of algorithms including Decision Tree, Random Forest, SVM, KNN, and Naive Bayes for predicting mobile price ranges, with tree-based ensembles consistently outperforming linear classifiers (ICMLT). More recent work has explored neural networks, with an MLP model achieving 92% accuracy in fair price prediction (Rezayan et al.), outperforming SVM and Logistic Regression baselines. This suggests deep learning represents a meaningful ceiling above tree-based methods and is a natural next step for this project.

A consistent limitation across the literature is the same one encountered here: hardware specifications alone do not fully explain price, and the absence of brand as a feature constrains model performance regardless of algorithm choice.

Project Objectives:

- **Clean and prepare** a real-world smartphone specifications dataset for machine learning by handling missing values, encoding categorical variables, and removing outliers.

-
- **Build and compare regression models** — Linear Regression, Ridge Regression, Polynomial Regression, Poly-Ridge Regression, and Random Forest — to predict smartphone prices, evaluated using R^2 under 5-fold cross-validation.
 - **Build and compare classification models** — Logistic Regression, K-Nearest Neighbors, Random Forest, and Naive Bayes — to assign devices to price tiers (Budget, Mid-Range, Premium, Flagship), evaluated using accuracy and classification reports.
 - **Identify the most predictive features** driving smartphone price through Random Forest feature importance.
 - **Visualize model performance**, feature relationships, and classification results to support interpretation of findings.

3. Materials and Methods

3.1 Data Description

For this project we used a “Smartphone Specifications Dataset” from Kaggle (<https://www.kaggle.com/datasets/devgondaliya007/smartphone-specifications-dataset>).

This dataset provides a comprehensive snapshot of the modern smartphone landscape, containing detailed specifications for nearly 1,000 unique smartphone models. We focused on predicting the price for the mobile phones from this data based on multiple factors like ‘clock_ghz’, ‘ram’, and ‘screen_size’.

3.2 Data Cleaning

- **Dropped high-missingness columns** — refresh_rate_hz, memory_card_max_gb, memory_card_type, display_type, fast_charge_w, sim_type, chipset, storage_gb, VoLTE.
- **Core type normalization** — Converted text labels (Octa, Hexa, Quad) to integers (8, 6, 4) via regex.
- **Resolution parsing** — Split resolution string (e.g. 1080x2400) into total_pixels (width × height), then dropped the original column.
- **Camera MP aggregation** — Parsed JSON-like rear_camera_mp_list strings and summed values into total_rear_camera_mp.
- **Missing value imputation** — Used a manually curated dictionary (sourced with the aid of Gemini) to fill missing clock_ghz, os, front_camera_mp, rear_camera_max_mp, and total_rear_camera_mp for specific named models.
- **Outlier removal** — Capped prices at ₹400,000 and RAM at 32 GB to remove unrealistic entries.
- **Encoding** — One-hot encoded os, converted NFC and ir_blaster booleans to integers (0/1).

3.3 Methodology

3.3.1 Scaling and normalization:

Before we started regression or classification, we noticed that our prices are heavily right skewed, which effects regression models negatively. So we log-transformed the data to

smoothen out the skewness.

Why We Transform Data: Visualizing the Target Log Transformation

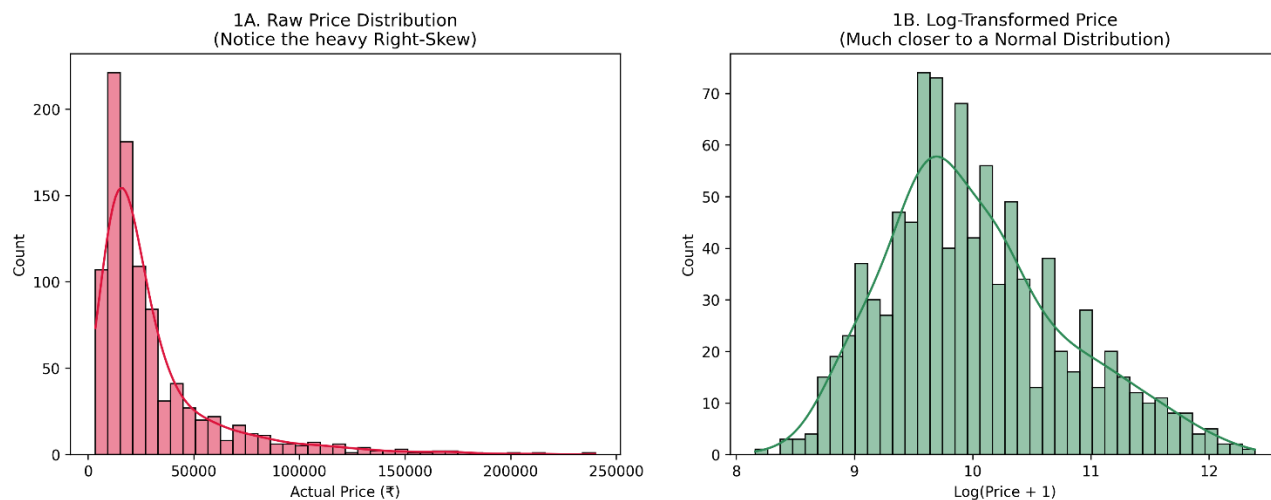


Figure 1 (Visualizing the Target Log Transformation)

And then all the columns were scaled using the *StandardScaler()* built-in function.

3.3.2 Regression models:

Multiple Linear Regression: Used as our baseline model, this maps the relationship between multiple smartphone features and price by assuming a straight-line trend. It calculates a constant "price-per-unit" for each feature. However, this assumes every spec increase adds equal value, which often misses the complex, non-linear pricing dynamics of premium smartphones.

Ridge regression: We implemented Ridge to ensure our baseline predictions generalized well, even without extreme multicollinearity. Standard Linear Regression can sometimes assign overly large weights to specific features, causing it to overreact to noise. Using 5-fold Cross-Validation, we found an optimal Alpha of 0.1 to apply a penalty that shrinks these weights. This prevents overfitting and yields much more reliable test performance.

Polynomial regression: We explored this to capture non-linear trends, like the exponential price jumps of flagship devices. By squaring and cubing features, it creates complex curves to fit the data. However, our Degree 2 and 3 results showed catastrophic overfitting. Without regularization, the model suffered from feature explosion creating extreme curves that perfectly memorized the training data but failed wildly on unseen test prices.

Poly-Ridge regression: After trying Polynomial regression and seeing it fail significantly on our test data for $d \geq 2$, we used Poly-ridge regression since it uses L2 regularization to

penalize the large coefficient weights that cause high-degree polynomial overfitting. This smooths the learned curve and significantly reduces the variance on our test data.

Random Forest Regression: When plotting some of our features against price, we could notice that our data is clustered around discrete values. Random Forest simply builds a flat "step" for each group. This allows it to perfectly capture the average price of each specific cluster without getting confused by the empty space between them or being pulled away by extreme outliers.

3.3.3 Classification models:

Naïve Bayes: Gaussian Naive Bayes was applied to classify smartphones into price brackets (Budget, Mid-Range, Premium) and rating brackets (High, Medium, Low) based on their hardware specifications. The model was trained using phone features such as RAM, battery capacity, camera quality, and screen characteristics. Numerical features were standardized so that large-value features did not dominate the calculations. The Gaussian version of Naive Bayes was selected because it works well with continuous numerical data by estimating the probability distribution of each feature within every class. However, despite its simplicity and fast computation, the model achieved relatively low accuracy because smartphone specifications are highly dependent on one another, while Naive Bayes assumes feature independence, limiting its ability to model the complex relationships within the dataset.

K-Nearest Neighbors Classification: After testing a simple Naive Bayes model, KNN was selected for the smartphone rating category classification task because it classifies phones based on similarity between their features. Phones with similar specifications such as RAM, battery capacity, camera quality, and price are likely to belong to the same rating category, making KNN well-suited for this dataset. Since KNN is distance-based, feature standardization was applied to ensure all features contributed fairly to the classification process. Different K values were evaluated using 5-fold cross-validation to find the best balance between overfitting and underfitting. After comparing multiple values, $K = 3$ achieved the highest average accuracy and was selected as the optimal number of neighbours for the final model.

Logistic Regression: We used a logistic regression model as a linear classifier to predict the probability of a phone belonging to a specific price tier (budget, mid, or premium), based on its hardware specifications. However, it has shown struggles when it comes to capturing complex & non-linear relationships within the dataset & using K-Fold Cross-Validation confirmed that the model had reached its performance limit.

Random Forest Classification: Random Forest was chosen because the smartphone market is too complex for a simple linear model like Logistic Regression. Features of “Premium” and “Mid-range” phones often overlap, making it difficult to separate categories using a single boundary. Instead of relying on one decision tree, Random Forest builds hundreds of trees where each tree learns from a different random subset of the dataset through bootstrapping, while also considering different random subsets of features. This makes every tree focus on different patterns in the dataset. The final prediction is made through majority voting across all trees, making the model more stable, accurate, and better at handling messy real-world data.

3.3.4 Data Visualization Techniques:

Box Plot: Box plots were used during the data preprocessing stage to analyse the distribution of numerical features such as price and rating. They helped identify outliers, extreme values, and the spread of the data, allowing abnormal entries to be handled appropriately. In addition, box plots were useful for determining suitable category boundaries when creating the rating and price classification groups.

Correlation Matrix: A correlation matrix was used to measure the strength and direction of linear relationships between all numerical variables in the dataset. This visualization helped identify which features were strongly related to one another and which variables had the greatest influence on target variables such as price and rating. It also assisted in feature selection and understanding potential multicollinearity between features.

Scatter Plot: Scatter plots were primarily used to evaluate regression model performance by comparing actual values against predicted values. By visualizing the relationship between predictions and true outputs, scatter plots made it easier to assess how closely the models fit the data, identify prediction errors, and observe whether the models followed the expected trend.

Confusion Matrix: Confusion matrices were used to evaluate the performance of the classification models. They provided a detailed breakdown of correct and incorrect predictions for each class, allowing analysis of which categories were classified accurately and which categories were commonly misclassified. This made it possible to assess the strengths and weaknesses of each classification model beyond overall accuracy alone.

Bar Graphs: Bar graphs were used to visually compare the performance metrics of different machine learning models. In regression tasks, they were used to compare training and testing R^2 scores across models, while in classification tasks they were used to compare model accuracies and the distribution of prediction categories. This made overall model performance comparisons clearer and easier to interpret.

3.4 Model Evaluation Metrics

3.4.1 K-Fold Cross-Validation:

Before calculating any final metrics, we implemented a 5-Fold Cross-Validation strategy. Rather than relying on a single, potentially biased train-test split, we aggregated out-of-fold (OOF) predictions for the entire dataset. This ensured that every single metric reported below was calculated strictly on unseen data, providing a highly reliable and unbiased estimate of how the models will perform in the real world.

(Figure 2) shows how a certain fold could influence our evaluation metric significantly, giving us false affirmation that our model is performing well.

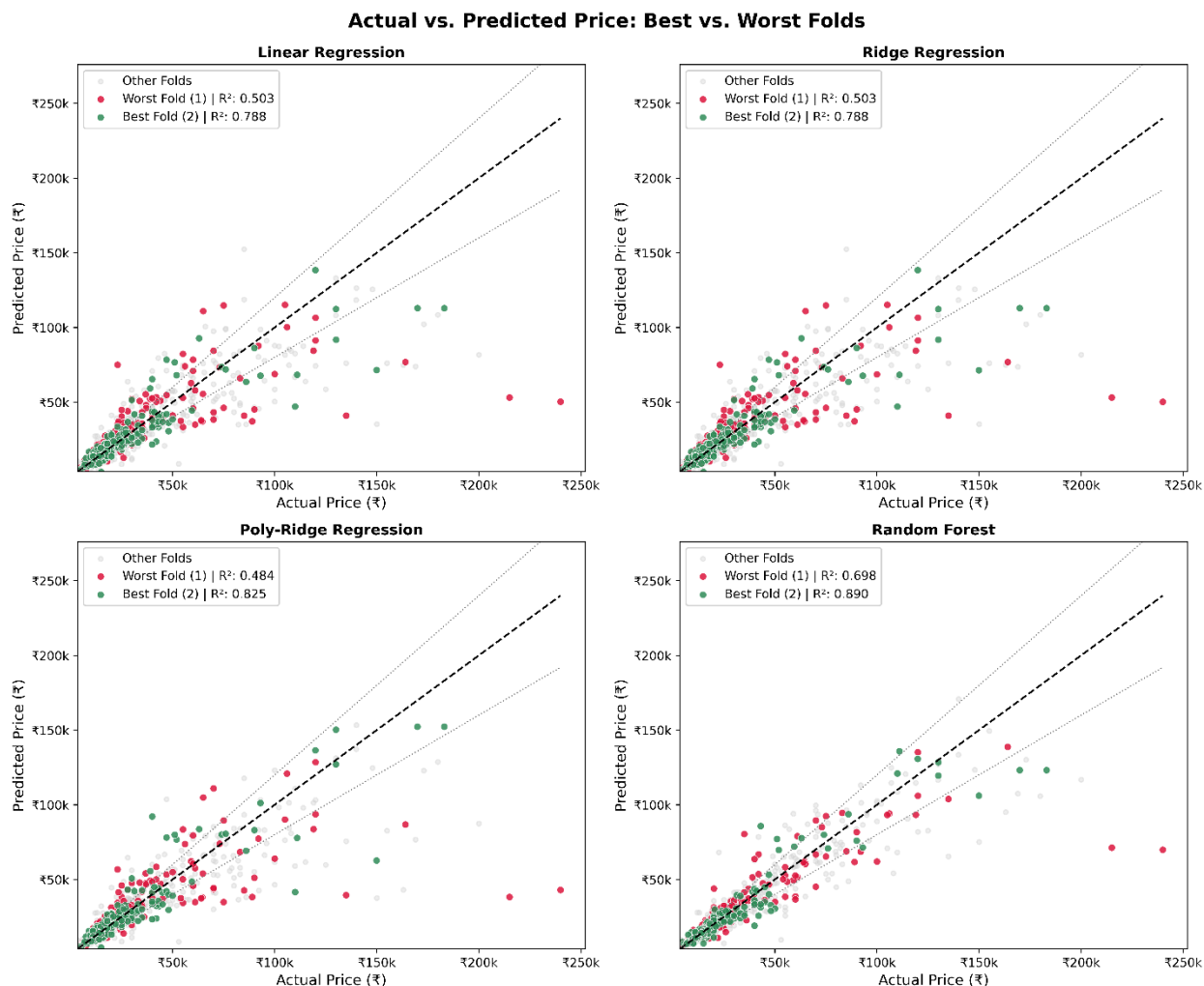


Figure 2 (Actual vs. Predicted Price: Best vs. Worst Folds)

3.4.2 Coefficient of Determination (R^2 Score):

We utilized the R^2 score as our primary evaluation metric for the regression models (Linear, Ridge, Polynomial, and Random Forest). This metric measures the proportion of the variance in the dependent variable (Price) that is predictable from the independent variables (Hardware Specifications).

3.4.3 Classification Accuracy:

For our Logistic Regression model, which was designed to predict discrete pricing tiers (Budget, Mid-Range, Premium) rather than continuous Rupee amounts, we relied on standard Classification Accuracy. This metric represents the straightforward percentage of phones that the model correctly sorted into their true respective market segments based on their hardware boundaries.

4. Results

4.1 Regression Findings and Performance

Using 5 fold cross validation with an 80/20 split. We applied 5 regression models to our test data.

4.1.1 Linear Regression

In our analysis, standard Linear Regression served as the fundamental baseline for predictive performance. The model achieved a moderate test R^2 of 0.61, confirming a foundational correlation between raw hardware specifications and retail value. However as seen in (Figure 3) it failed to predict the price of Premium, high priced, phones.

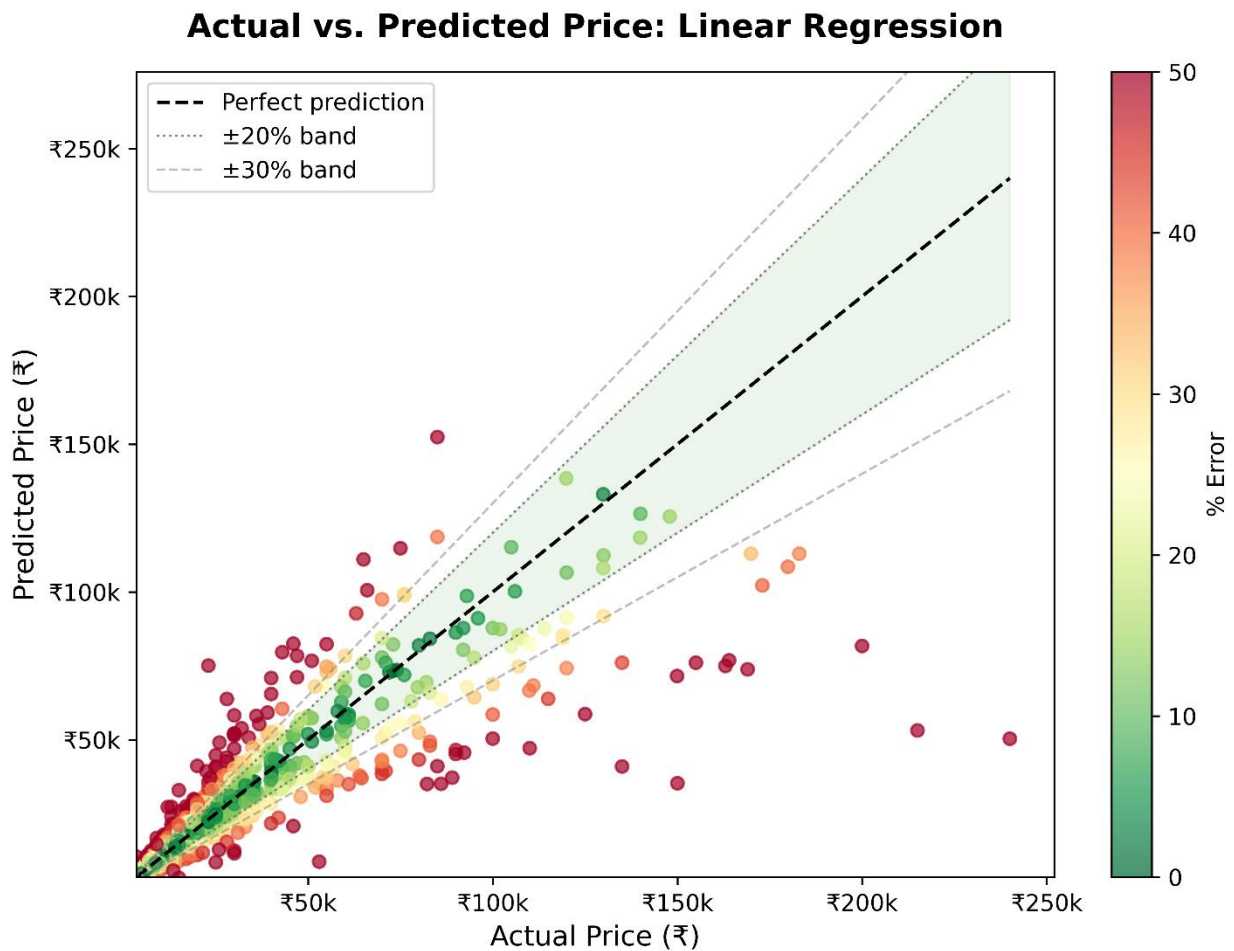
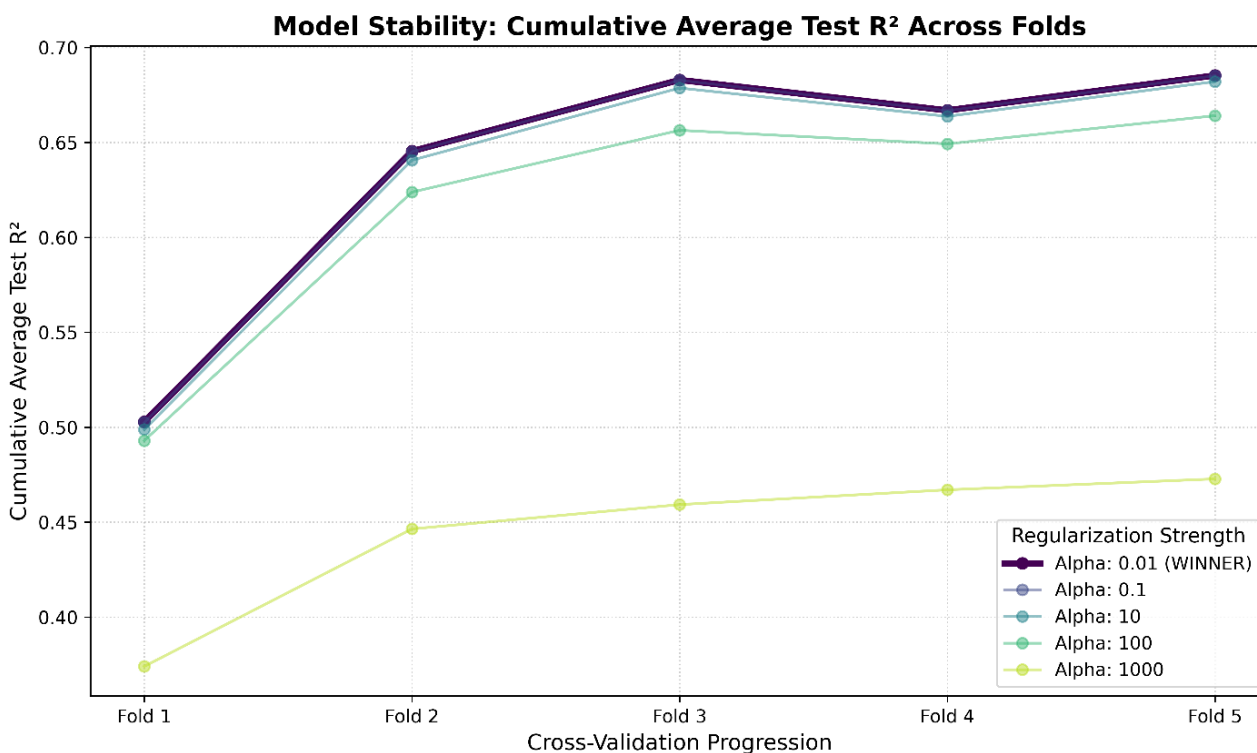


Figure 3 (Actual vs. Predicted Price: Linear Regression)

4.1.2 Ridge Regression

We applied 5 fold cross validation for each of the 5 standard alpha values to find the best one for our model. Figure (4) demonstrates high stability across a wide range of regularization strengths. The validation trajectories for Alphas 0.01, 0.1, and 10 are nearly



indistinguishable, hovering around an R^2 of 0.68. This tight clustering indicates that the pre-processed dataset does not suffer from high multicollinearity or explosive coefficients that require heavy penalization.

Figure 4 (Cumulative Average Test R^2 Across Folds)

4.1.4 Poly-Ridge Regression

To capture the non-linear pricing jumps in the smartphone market, we evaluated a Degree-2 Polynomial Regression model. However, this approach clearly demonstrated the dangers of unconstrained mathematical complexity. While the pure polynomial model attempted to fit the training data, it catastrophically overfit the localized noise, resulting in wildly inaccurate predictions and a negative R^2 score on unseen test data. To resolve this instability, we introduced L2 regularization, creating a Poly-Ridge model. By applying a strict mathematical penalty to the extreme polynomial coefficients, the Poly-Ridge model

successfully smoothed and stabilized the non-linear curves. This regularized approach yielded a highly reliable test R^2 of 0.77.

4.1.5 Random Forest Regression

As theorized in our methodology, the Random Forest model successfully adapted to the highly clustered nature of the smartphone market. Figure (5) visually demonstrates this advantage by comparing the predictions for Total Pixels and Clock Speed against actual prices. While the Linear Regression baseline (red dashed line) blindly slices through the middle of the dense vertical clusters over-predicting budget phones and under-predicting flagships the Random Forest model (solid blue line) dynamically steps up and down to map the localized reality of each hardware tier. This structural flexibility directly translates to superior variance capture, outperforming the linear baseline by over 25% in the Total Pixels scenario (R^2 : 0.680 vs. 0.422).

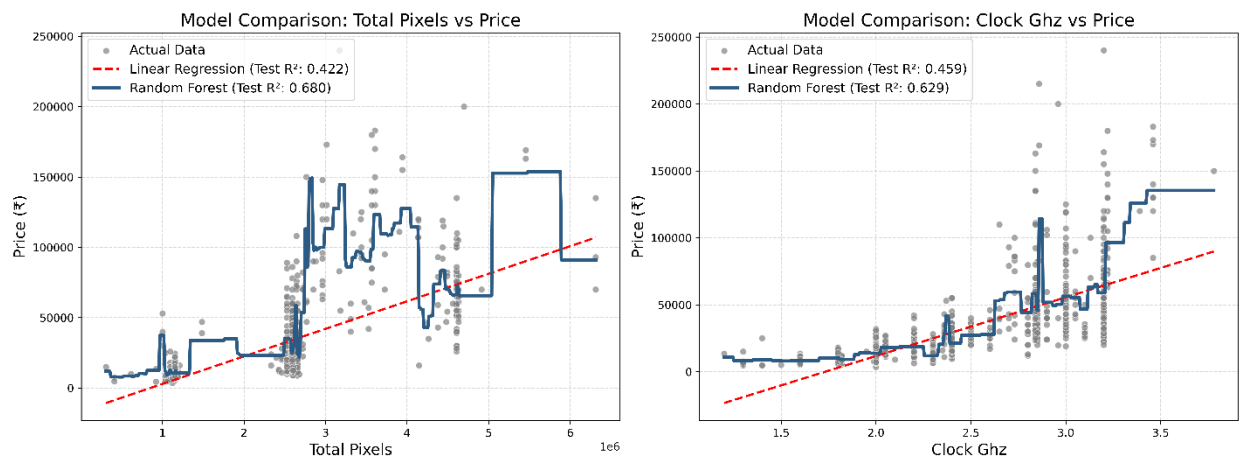


Figure 5 (Pixels and GHz vs. Price)

4.1.6 Regression – Model Performance

Model Behaviors: Linear and Ridge Regression perform almost identically, indicating a stable baseline. The Random Forest model shows the tightest clustering along the perfect prediction line for budget and mid-range devices. However it also struggles with outlier

data as shown by the red dots. Overall this helps visualize where the R^2 values come from.

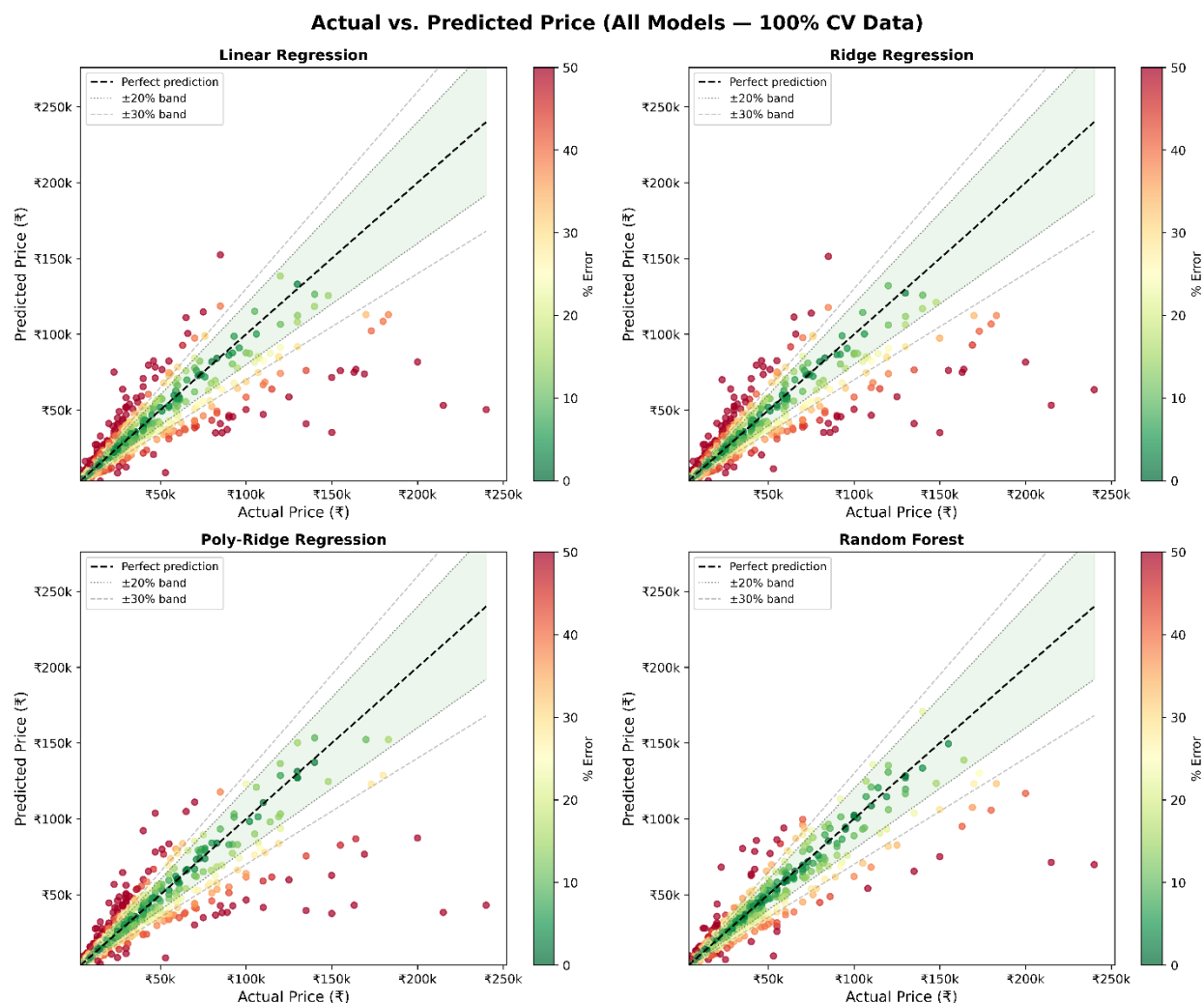


Figure 6 (Actual vs. Predicted All Models)

Figure (7) a clear trade-off between complexity and generalization. While the Linear baseline (R^2 : 0.69) was moderate, Ridge Regression (R^2 : 0.70) improved stability by preventing coefficient explosion. Standard Polynomial Regression catastrophically overfit and failed on test data but combining it with L2 regularization (R^2 : 0.71) successfully balanced non-linear curves with stable predictions. Ultimately, Random Forest emerged as the superior model (R^2 : 0.83), as its step-based approach easily handled the smartphone

market's natural price clusters and outliers without being misled.

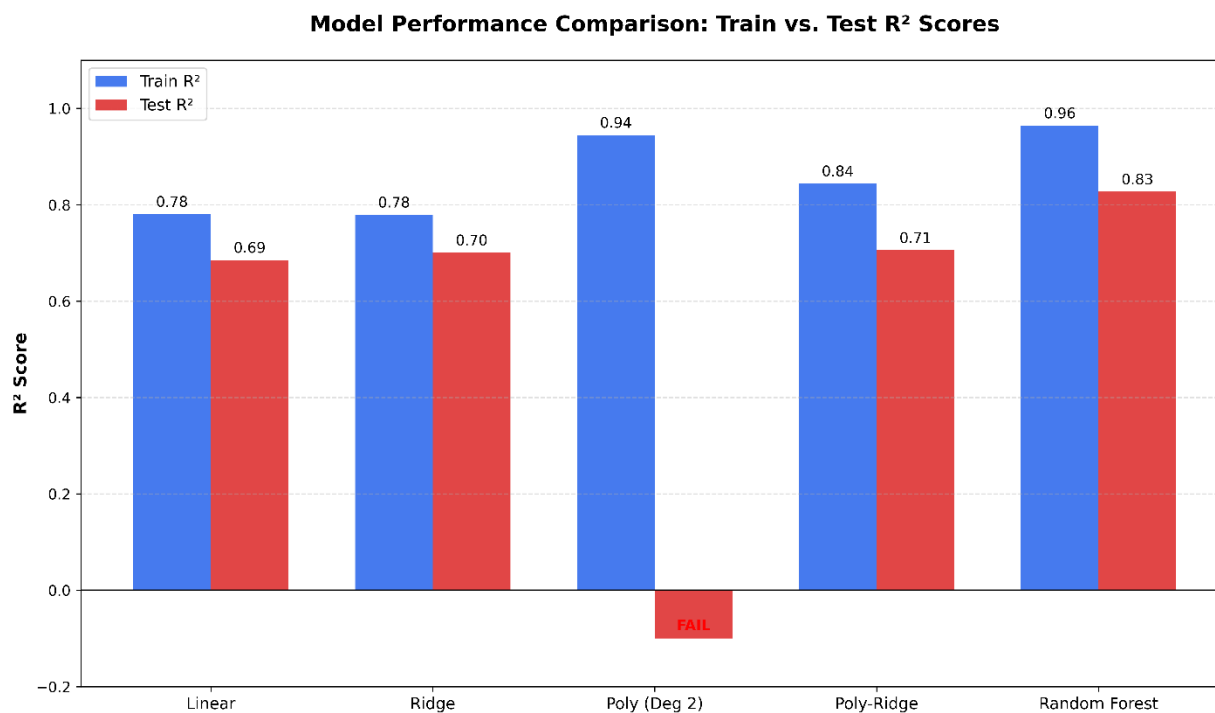


Figure 7 (Model Performance Comparison: Train vs. Test R^2)

4.2 Classification Findings and Performance

4.2.1 Logistic Regression

To evaluate our starting point, we analysed the normalised confusion matrix for the Logistic Regression Baseline (Figure 8), which achieved a mean accuracy of 76.05%. The results illustrate where a linear approach struggles. While it identifies the Mid-range category with 84% accuracy, it faces significant challenges with the other tiers. Specifically, 34% of Budget devices were misclassified as Mid-range, and the model showed similar confusion with the Premium tier, incorrectly tagging 29% of high-end phones as Mid-range. This visualisation proves that without advanced ensemble techniques, a standard linear model tends to pull predictions toward the majority class whenever it encounters hardware overlaps. This performance serves as the floor for our project, highlighting the necessity of the Random Forest model to accurately distinguish the nuanced specifications that separate Budget and Premium phones from the rest of the market.

Logistic Regression: Where the Linear Model Fails

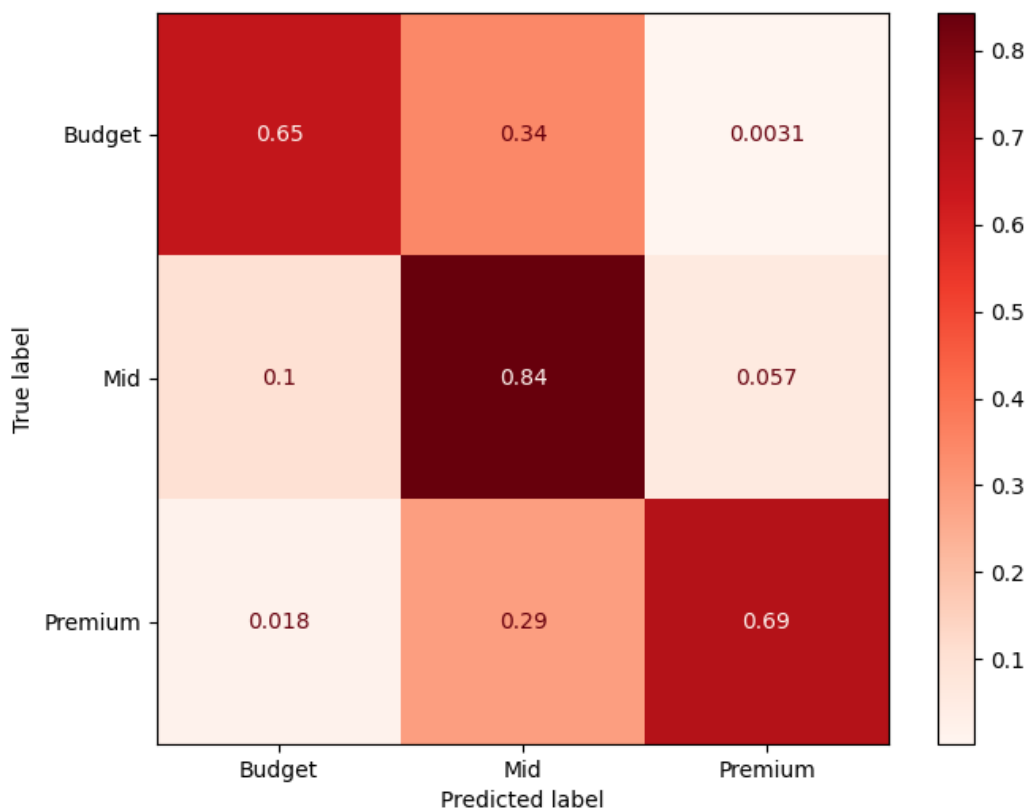


Figure 8 (Logistic Regression: Where the Linear Model Fails)

This visualisation highlights a core challenge within the dataset: Feature Overlap. By plotting two key drivers (RAM & Rating), we can see that while price categories generally trend upward, they do not form isolated clusters. Significant "crowding" occurs where Budget and Mid-range devices share similar RAM capacities, & Mid-range and Premium devices overlap at higher specs. This mix explains why the Logistic Regression peaked at 76.05%; a simple linear boundary cannot cleanly separate these overlapping points without making frequent errors. This finding justifies our shift to the Random Forest model, which can zigzag through these dense, overlapping regions to identify non-linear patterns that a straight line simply cannot capture.

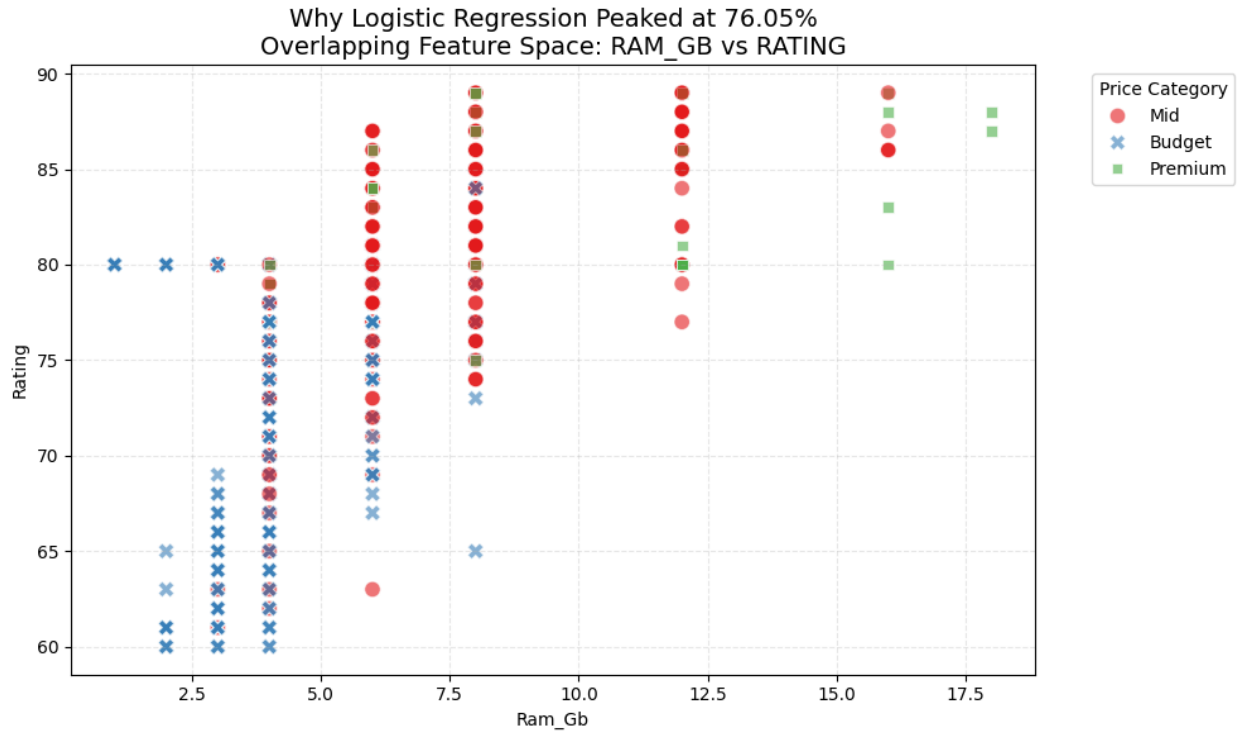


Figure 9 (Overlapping Feature Space: RAM_GB vs. RATING)

4.2.2 Random Forest Classification

The Random Forest classification demonstrates strong performance, achieving an accuracy of 86.32%, which significantly outperforms the 76.05% Logistic Regression baseline. The model was evaluated using precision (the proportion of predicted labels that are correct), recall (the proportion of actual cases correctly identified), & the F1-score (the harmonic mean balancing precision and recall), while support indicates the number of actual samples in each category. The Mid-range tier emerged as a top performer with an F1-score of 0.87, supported by the largest sample size of 95 devices. Notably, the Budget category achieved the highest precision (0.94), meaning that when the model predicted a "Budget" label, it was correct 94% of the time. Conversely, the Premium tier saw the highest recall (0.95), successfully catching nearly every high-end device despite a smaller support of 19. These balanced metrics confirm that the Random Forest ensemble is highly effective at tiering smartphones based on their specific hardware patterns. The weighted average mathematically balances the high performance of the dominant Mid-range category with the smaller Budget and Premium tiers to ensure the final scores reflect the entire dataset's reality.

=== Model performance ===

Accuracy: 0.8632

=== Classification Report ===

	Precision	Recall	F1-score	support
Budget	0.938462	0.802632	0.865248	76.0
Mid	0.841584	0.894737	0.867347	95.0
Premium	0.750000	0.947368	0.837209	19.0
Weighted avg	0.871177	0.863158	0.863494	190.0

Table 1 (Random Forest Classification Model Performance)

The confusion matrix (Figure 10) illustrates that the Random Forest model maintained high accuracy across all tiers, with most predictions falling correctly along the main diagonal. In this iteration, the model was most effective at identifying Mid-range devices with 85 correct predictions, followed by 61 for Budget and 18 for Premium. The matrix also highlights that the most frequent errors occurred between adjacent categories, specifically with 15 Budget devices being misclassified as Mid-range. These results confirm that while the model is highly capable of distinguishing between price tiers, the "blurred boundaries" in hardware specifications are most prominent between the entry-level & mid-range segments.

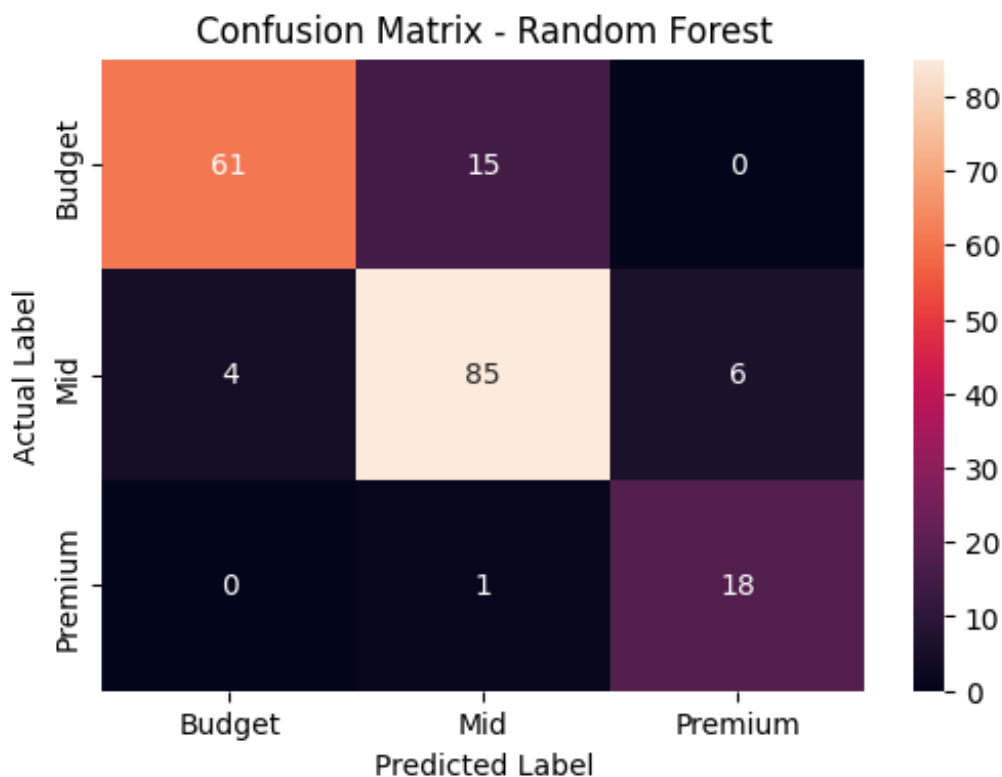


Figure 10 (Confusion Matrix - Random Forest)

To compare the actual & predicted distributions of smartphone price categories, a grouped bar graph was utilised (Figure 11) to visualise the Random Forest model's alignment with the ground truth. Across all three categories, the orange "Predicted" bars closely mirror the blue "Actual" bars, demonstrating the model's ability to maintain the overall balance of the dataset without significant overprediction of any single tier. While the Mid-range category leads in total volume, followed by Budget and Premium, the slight inconsistencies (such as a minor under-prediction in the Budget tier) reflect the hardware overlaps identified earlier. Overall, the strong consistency between these distributions confirms that the Random Forest successfully captured the distinct patterns of each price bracket while avoiding systematic bias.

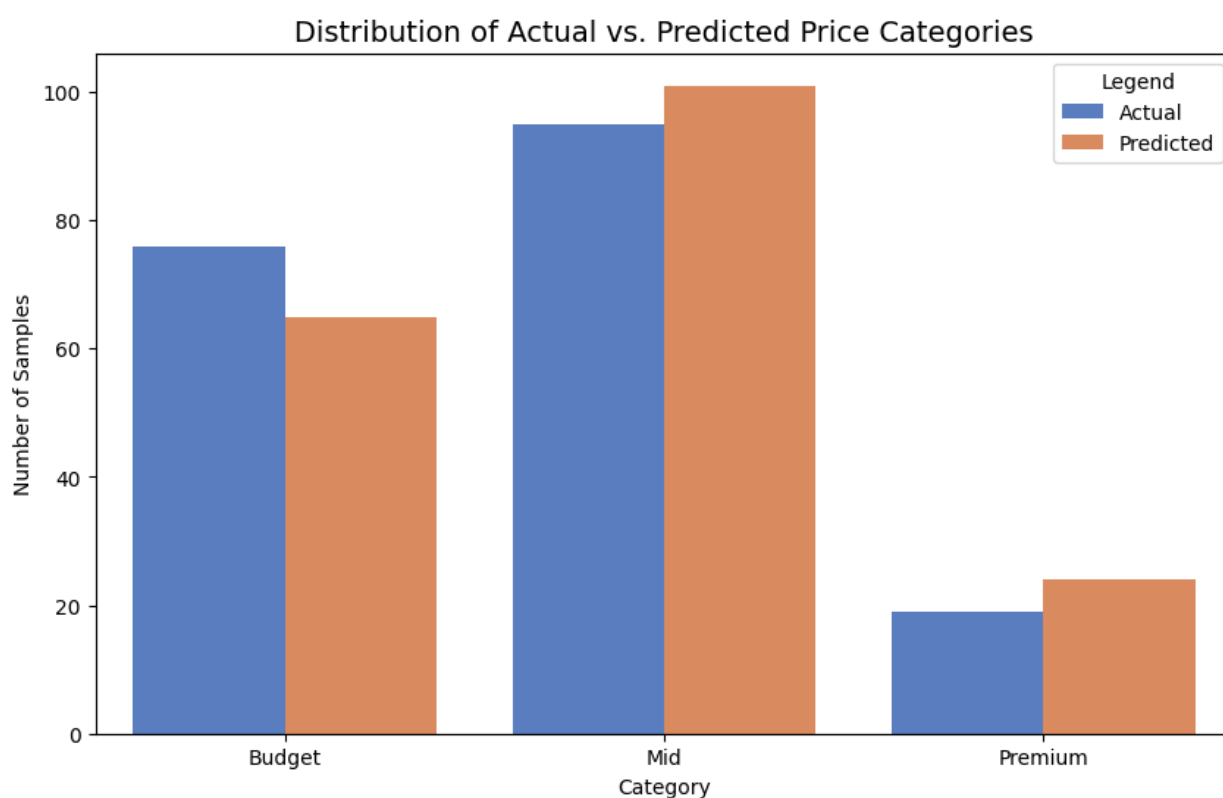


Figure 11 (Distribution of Actual vs. Predicted Price Categories)

4.2.3 Naïve Bayes Classification

After establishing the Logistic Regression baseline & the Random Forest improvement, we explored a third approach using a Gaussian Naive Bayes classifier. This model operates on the principle of probability, if every feature (like RAM or Battery) contributes independently to the final price category.

The Naive Bayes model achieved an overall accuracy of 78.42%, placing it firmly between the logistic regression & random forest classification models. When looking at the class-

specific metrics, the model excelled in the Mid-range tier with a high recall of 0.92, suggesting it is very effective at catching most common devices. However, the model struggled significantly with the Premium tier, recording a recall of only 0.32. This indicates that while the model was precise when it did guess premium (0.88 precision), it missed over two-thirds of high-end devices, misclassifying them as other categories.

=== NAIVE BAYES PRICE BRACKET EVALUATION ===

Accuracy: 0.7842

	Precision	Recall	F1-score	support
Budget	0.87	0.72	0.79	65
Mid	0.74	0.92	0.82	103
Premium	0.88	0.32	0.47	22

Table 2 (NAIVE BAYES PRICE BRACKET EVALUATION)

The visualisation (Figure 12) provides a dual perspective on the Naïve Bayes classification task, contrasting the distribution of smartphone price brackets against the resulting confusion matrix for price prediction. The bar chart reveals a significantly imbalanced dataset, where the Mid-range category dominates with over 500 samples, while the Premium tier represents the smallest subset at approximately 100 samples. This distribution is mirrored in the confusion matrix, where the model shows the highest confidence in the Mid-range category, correctly identifying 95 devices. However, a pattern is visible where most misclassifications occur between neighbouring tiers, such as the 18 Budget phones mistaken for Mid-range & the 15 Premium phones also mistaken for Mid-range. This suggests that while the model is generally robust, the hardware specifications at the boundaries of these price brackets share enough similarities to occasionally blur the classification lines, particularly in the underrepresented Premium tier.

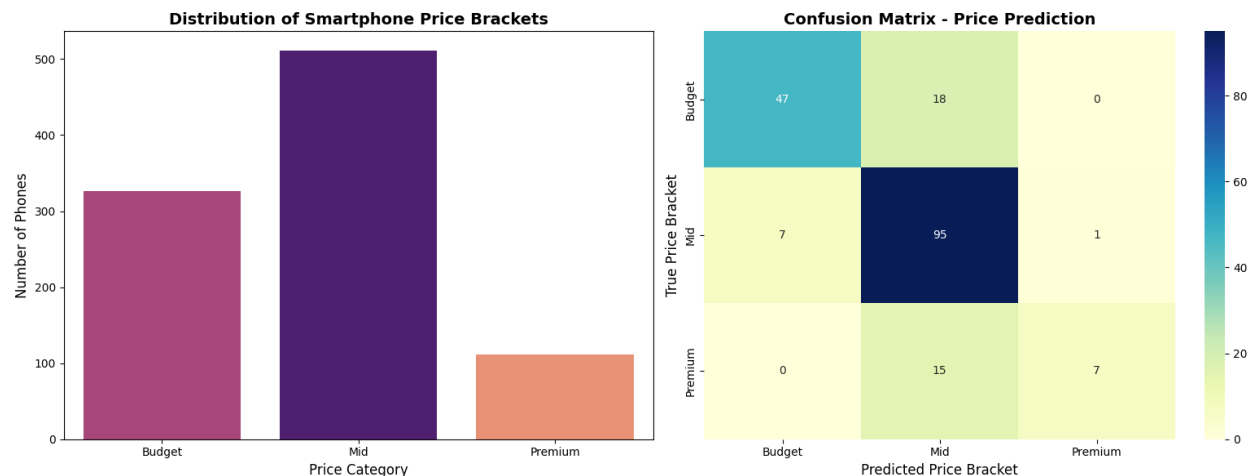


Figure 12 (dual perspective on the Naïve Bayes classification task)

For the second classification task, the objective was to classify smartphones into rating categories (High, Medium, and Low) based on their user ratings. As a baseline approach, Gaussian Naive Bayes was selected due to its simplicity, fast computation time, and suitability for numerical data. Standard rating thresholds were initially adopted, where ratings above 75 were classified as High, ratings between 50 and 75 as Medium, and ratings below 50 as Low.

Using these thresholds, the Gaussian Naive Bayes model achieved a test classification accuracy of 32%, which indicates poor classification performance for a three-class problem. Evaluation metrics in the table below further showed severe class imbalance, as the dataset contained a disproportionately large number of highly rated smartphones while the Low rating category contained almost no samples as shown in Figure 13. As a result, the model struggled to generalize effectively across all classes and tended to bias predictions toward the dominant categories.

The evaluation metrics reveal that the Gaussian Naive Bayes model performed inconsistently across the rating categories. For the High category, the precision score of 1.00 indicates that whenever the model predicted a phone as High-rated, the prediction was always correct. However, the recall score of only 0.13 shows that the model was able to identify just 13% of the actual High-rated phones, meaning that the majority of High-rated phones were incorrectly classified into other categories. This resulted in a low F1-score of 0.23, indicating poor overall performance for this class despite the perfect precision.

For the Medium category, the recall score of 1.00 indicates that all actual Medium-rated phones were successfully identified by the model. However, the precision score dropped significantly to 0.25, meaning that many phones predicted as Medium were actually from other categories. The F1-score of 0.40 reflects this imbalance between precision and recall. Additionally, the support values show a major class imbalance within the dataset, with 147 High-rated phones compared to only 43 Medium-rated phones and no Low-rated phones. This uneven distribution contributed heavily to the poor generalisation ability of the model and its unstable classification behaviour.

	Precision	Recall	F1-score	support
High	1.00	0.13	0.23	147
Medium	0.25	1.00	0.40	43

Table 3 (Gaussian Naive Bayes Model Performance)

Confusion Matrix - Naive Bayes Rating Classification

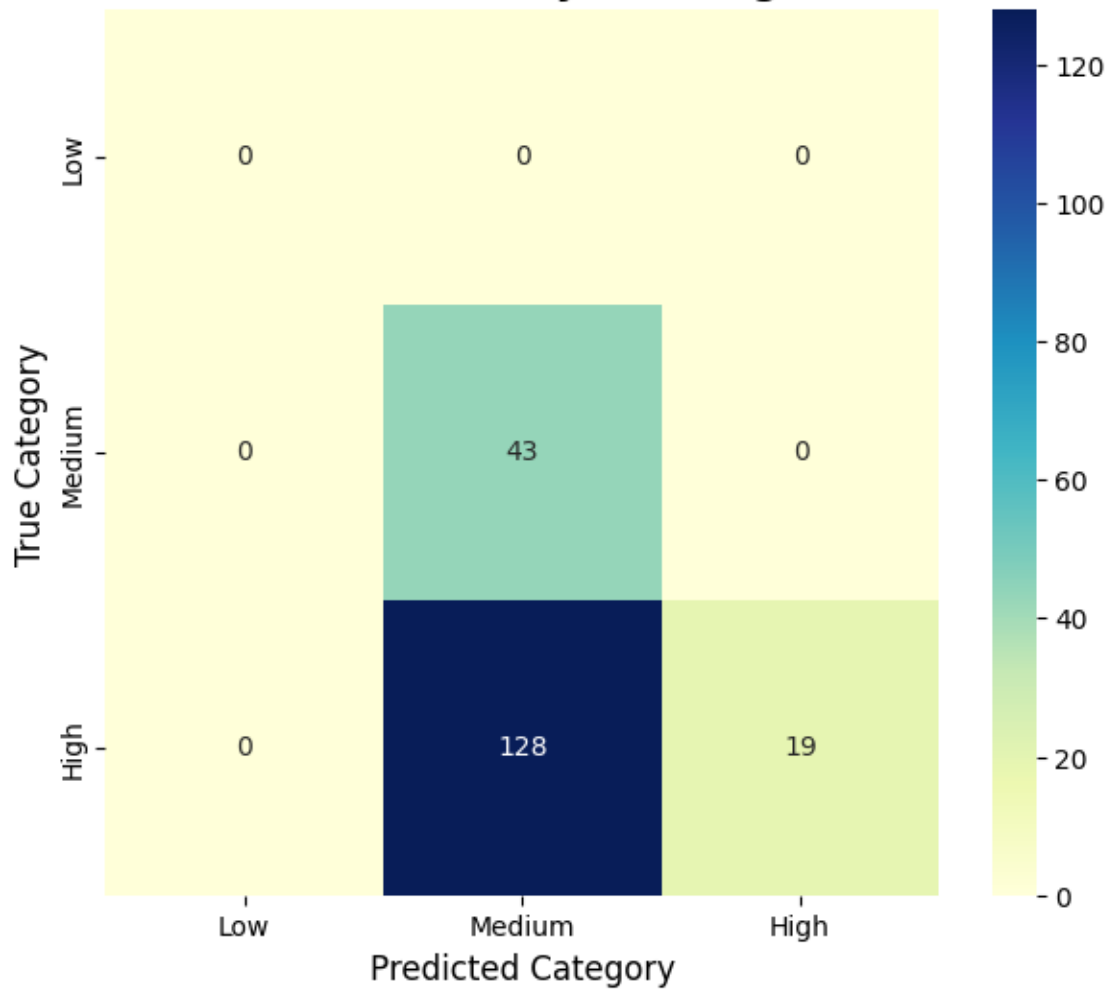


Figure 13 (Confusion Matrix - Naive Bayes Rating)

Another major limitation was that Gaussian Naive Bayes assumes that the features follow a Gaussian (normal) distribution. However, analysis of the data set shown in (Figure 14) revealed that rating distributions were heavily skewed rather than normally distributed.

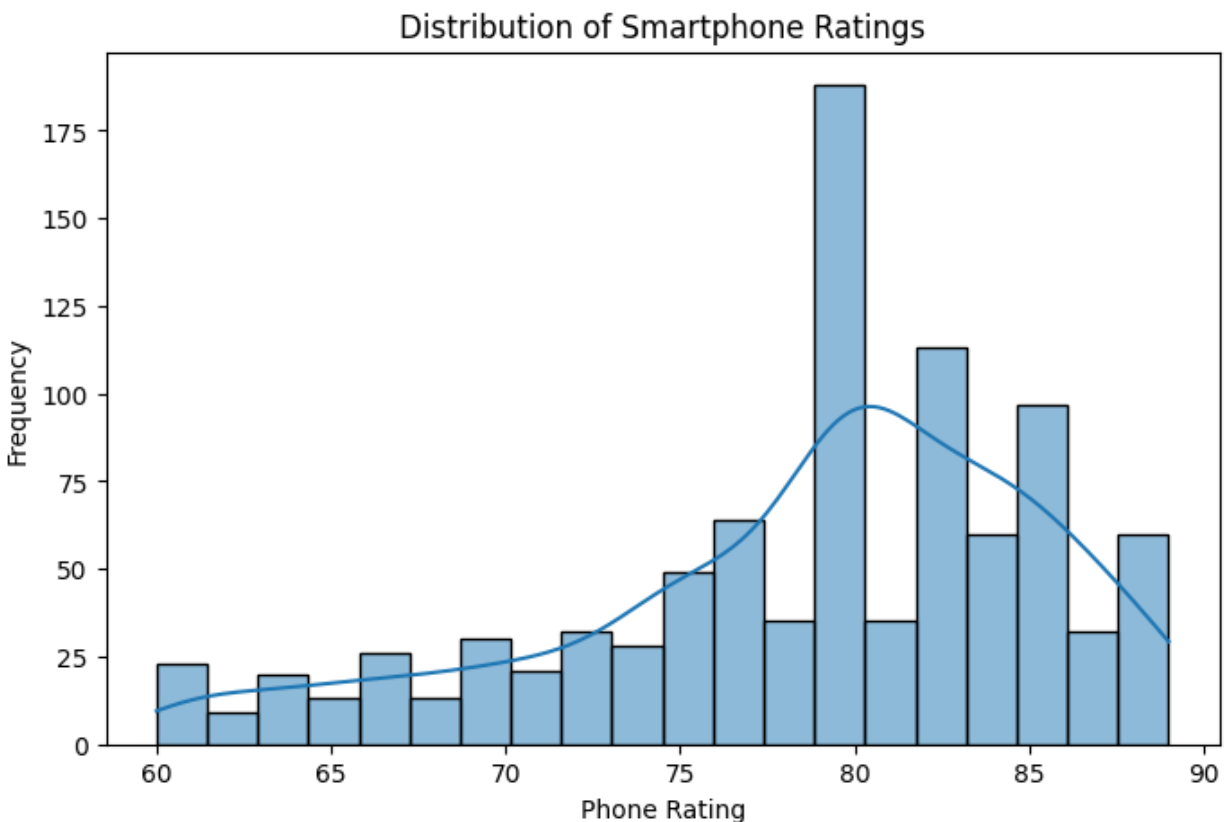


Figure 14 (Distribution of Smartphone Ratings)

This heavy left skew violated the underlying assumptions of the model and reduced its ability to accurately estimate class probabilities. Consequently, the Gaussian Naive Bayes classifier was determined to be unsuitable for this dataset, motivating the transition to more flexible classification approaches such as K-Nearest Neighbours (KNN).

Before implementing the K-Nearest Neighbours model, it was necessary to address the left-skewed distribution of the dataset to ensure more balanced class representation. This was achieved by analysing the data using a box plot (Figure 15), which provided insight into the distribution of rating values and helped identify more appropriate category boundaries. Based on this analysis, adjusted thresholds were defined to improve class balance, with approximately 83 used as the boundary for the High category and 78 for the Medium category. These revised thresholds resulted in a more evenly distributed classification

structure, improving the quality and reliability of the subsequent model training process.

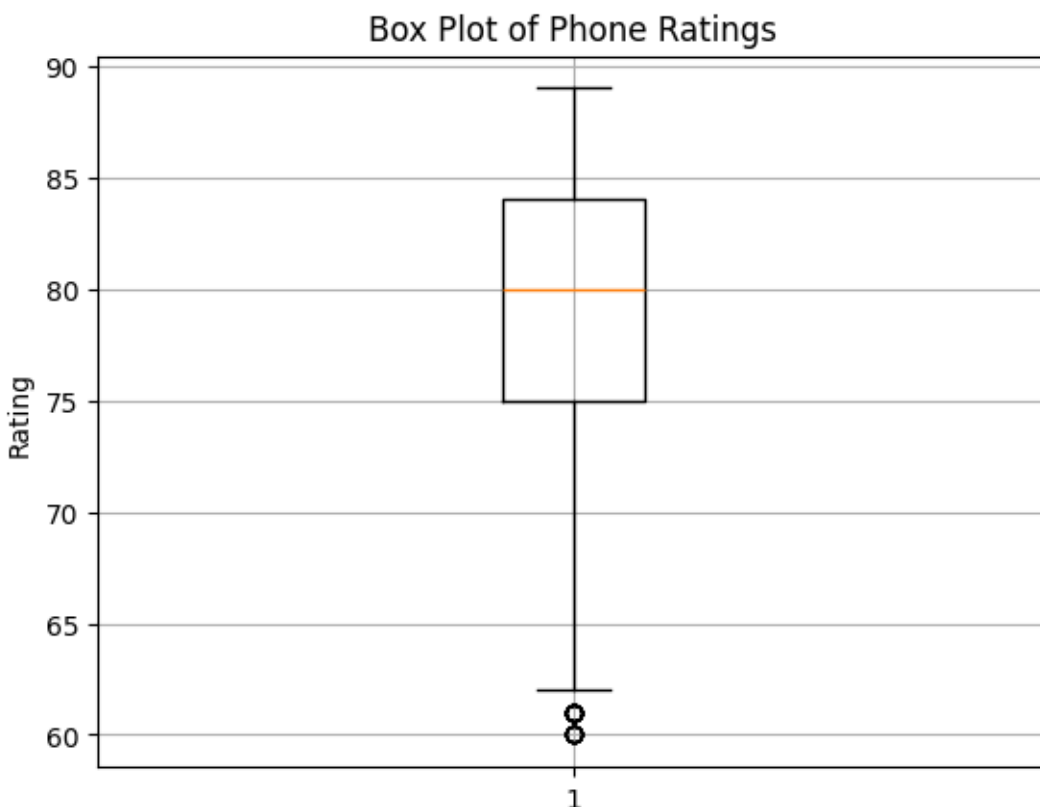


Figure 15 (Box Plot of Phone Ratings)

By adjusting the classification thresholds, the model's performance improved significantly, with accuracy increasing from 32% to 48%. The corresponding confusion matrix (Figure 16) reflects this improvement, showing a more balanced distribution of correct predictions across the classes. However, despite this progress, the overall performance remained insufficient for reliable classification, as notable misclassifications were still present. As a result, a more suitable algorithm was explored, leading to the adoption of the K-Nearest Neighbours model for improved performance.

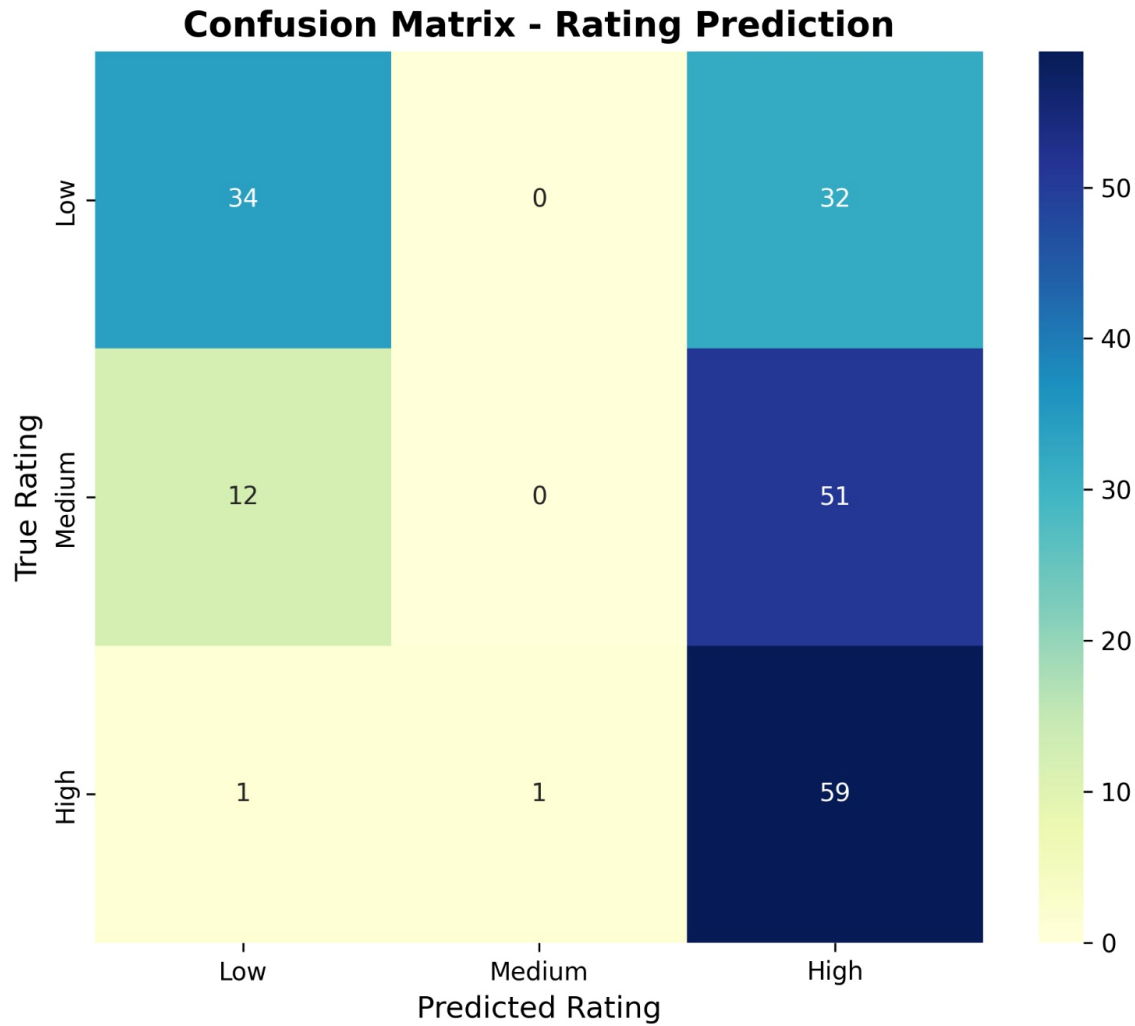


Figure 16 (Confusion Matrix - Rating Prediction)

4.2.4 K-Nearest Neighbors Classification

After defining the updated rating thresholds, the next step was to determine the optimal value of K for the K-Nearest Neighbours algorithm. To achieve this, multiple odd values of K ranging from 3 to 9 were evaluated. Odd values were selected to avoid tie situations during majority voting in classification.

Very small values such as $K = 1$ were avoided as they make the model highly sensitive to noise and outliers, leading to unstable predictions. On the other hand, larger values beyond 9 were found to overly generalize the data, resulting in underfitting and reduced model sensitivity to meaningful patterns within the dataset. By testing a range of

intermediate values, the model performance was compared systematically, and the optimal K was selected based on cross-validated accuracy.

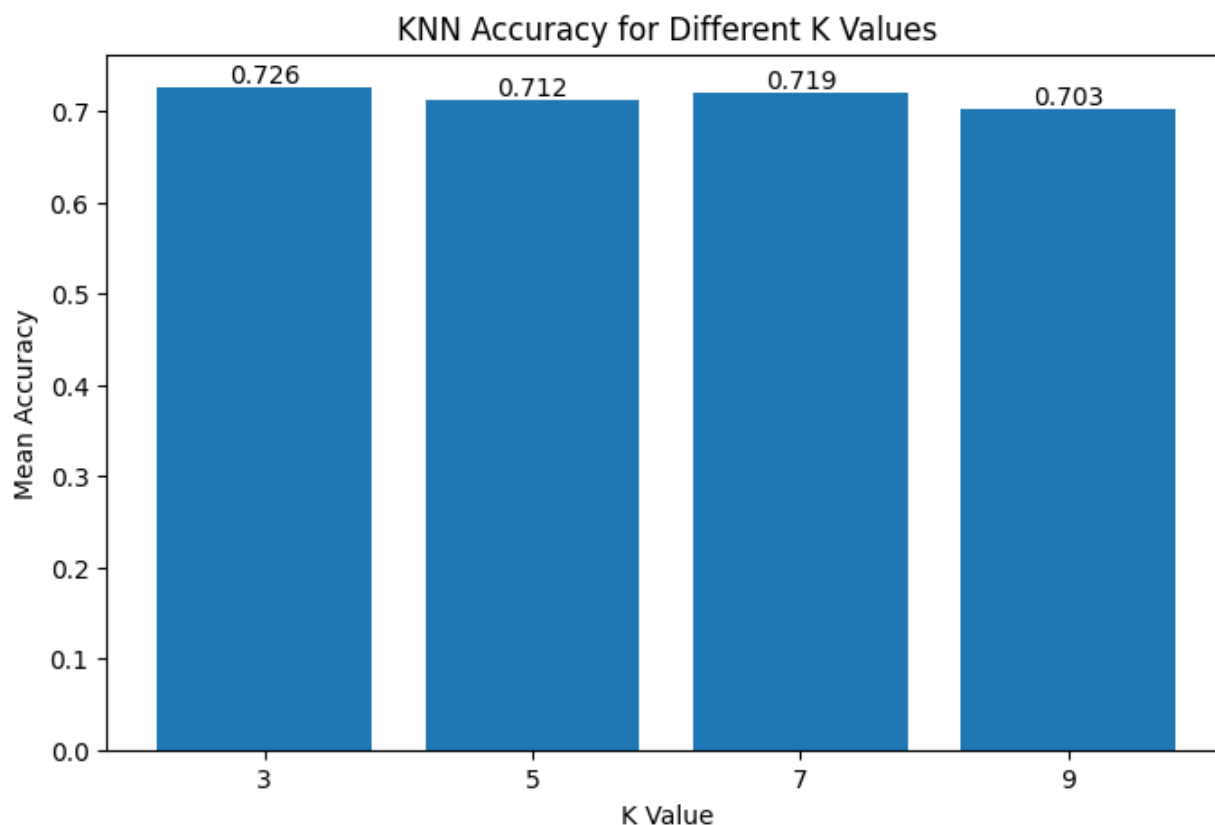


Figure 17 (KNN Accuracy for Different K Values)

As shown in (Figure 17), the classification accuracy obtained for the different K values were relatively similar. However, the highest average accuracy was achieved when $K = 3$, and therefore this value was selected as the optimal number of neighbors for the final K-Nearest Neighbors model. To ensure a more reliable and robust evaluation, 5-fold cross-validation was applied for each tested K value. This approach allowed the model to be trained and evaluated on multiple subsets of the dataset, reducing the impact of random data splitting and providing a more representative estimate of model performance, particularly given the relatively limited test dataset of approximately 190 smartphones.

The evaluation metrics in (Table 4) show that the K-Nearest Neighbors model achieved a significantly more balanced performance across all rating categories compared to the previous Gaussian Naive Bayes model. For the High category, the model achieved a precision of 0.70, meaning that 70% of the phones predicted as High-rated were correctly classified. The recall score of 0.77 indicates that the model successfully identified 77% of

all actual High-rated phones. This resulted in an F1-score of 0.73, demonstrating a strong balance between precision and recall for this category.

For the Medium category, the precision, recall, and F1-score were all 0.63, showing that the model achieved moderate performance when classifying medium-rated phones. This lower performance is expected because Medium-rated phones often share specifications with both High and Low categories, creating overlap between classes and making them more difficult to separate clearly.

For the Low category, the model achieved the best overall performance, with a precision of 0.83 and a recall of 0.76. This indicates that the classifier was highly effective at distinguishing low-rated phones from the other categories while still correctly identifying most of the actual Low-rated samples. The resulting F1-score of 0.79 reflects strong and stable classification performance.

Additionally, the support values show that the dataset was relatively balanced across all three categories, with 61 High-rated phones, 66 Low-rated phones, and 63 Medium-rated phones in the testing set. This balanced distribution contributed to the improved stability and fairness of the KNN classifier across all classes.

	Precision	Recall	F1-score	support
High	0.70	0.77	0.73	61
Medium	0.63	0.63	0.63	63
Low	0.83	0.76	0.79	66

Table 4 (K-Nearest Neighbors Model Performance)

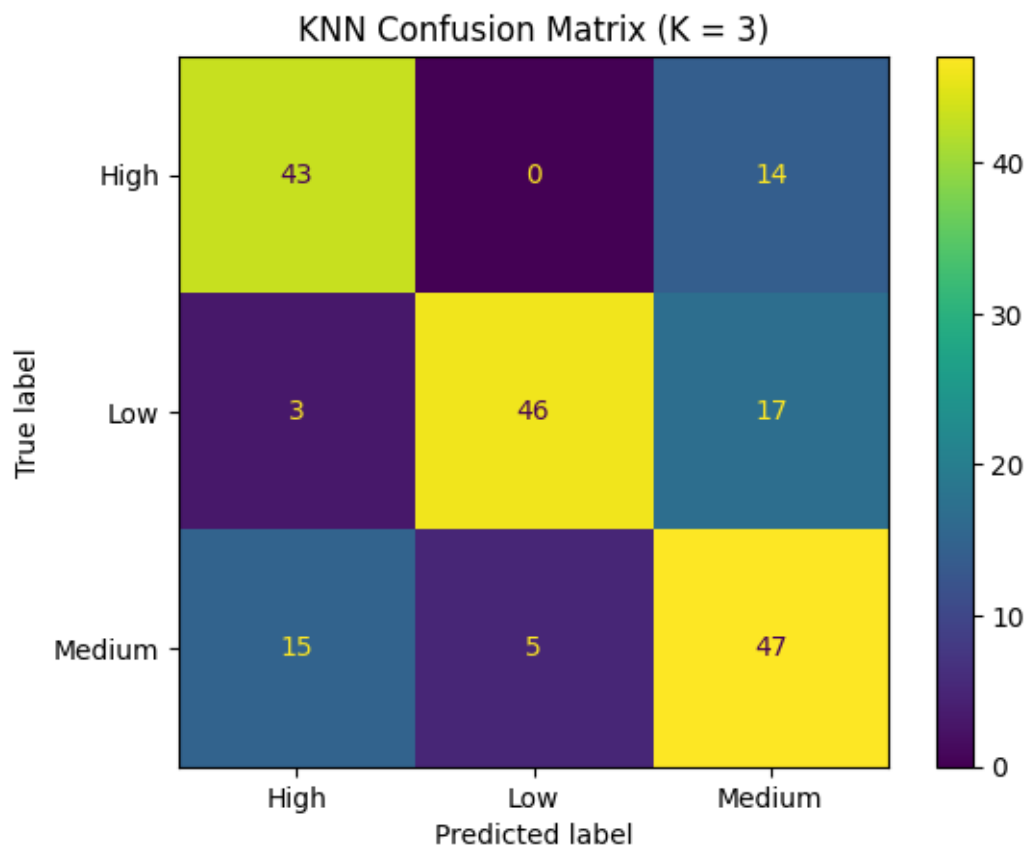


Figure 18 (Confusion Matrix - K-Nearest Neighbors Model)

(Figure 18) presents the confusion matrix for the final K-Nearest Neighbours model using $K = 3$, which was selected as the optimal parameter based on cross-validation results. The matrix illustrates the distribution of correct and incorrect predictions across the High, Medium, and Low rating categories, providing a detailed view of the model's classification performance. Overall, the results show a strong concentration of correct predictions along the diagonal, indicating that the model is effectively distinguishing between the different rating classes, with only a limited number of misclassifications across categories.

4.2.5 Pricing Category Classification – Model Performance

The Model Performance Comparison chart illustrates a clear step-up in predictive power as we move from basic models to more advanced techniques. The Random Forest is the standout winner, maintaining a strong and steady lead across all four metrics with an accuracy of 0.86 and a balanced 0.86 F1-score. This high performance across the board proves that the model is very reliable at identifying all price tiers correctly without being biased toward just one specific category. In contrast, Naive Bayes serves as a middle-tier option; while it shows a visible improvement in Precision (0.80) compared to the baseline, it still lacks the overall consistency of the random forest approach. The Logistic Regression

model sits at the bottom of the group, with scores hovering around 76% for every metric, which indicates that a simple linear strategy isn't enough to handle this specific dataset. Ultimately, this graph confirms that the Random Forest's ability to analyse complex relationships between features makes it the best choice for separating smartphone price brackets where hardware specifications often overlap.

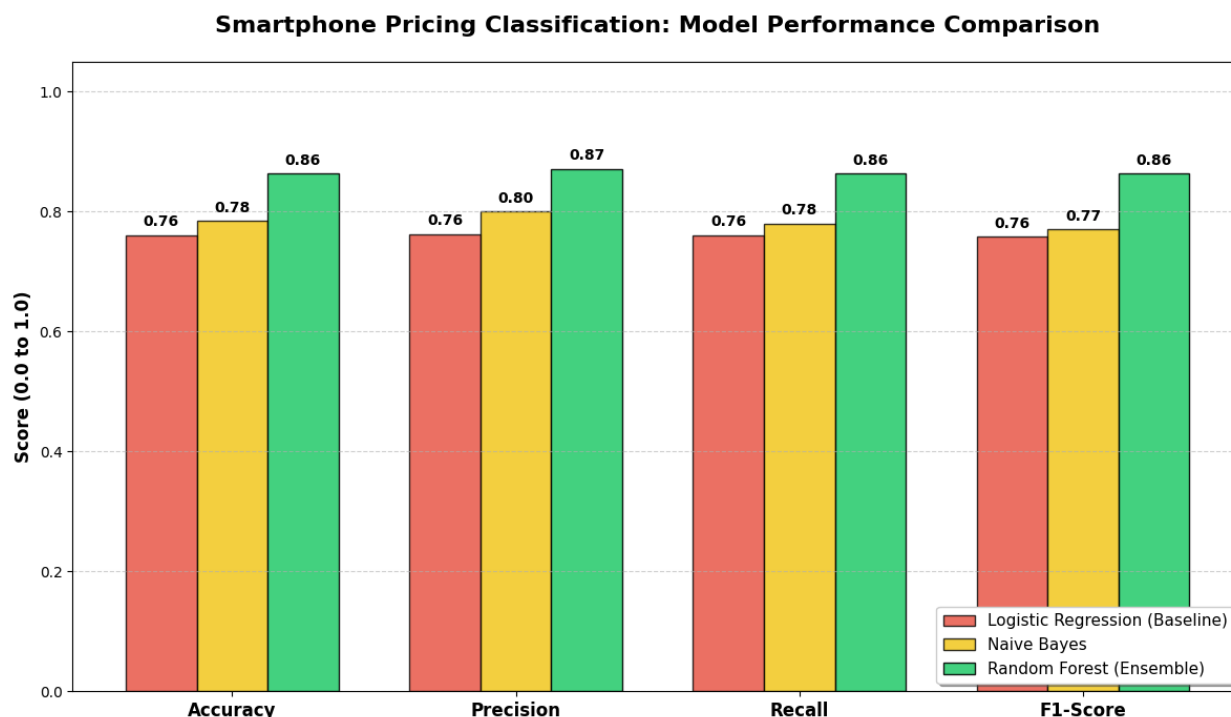


Figure 19 (Smartphone Pricing Classification: Model Performance Comparison)

4.2.6 Rating Category Classification – Model Performance

After training and evaluating both classification models using the same rating thresholds, the average accuracy, precision, recall, and F1-score obtained from each model are

presented in (Figure 20)

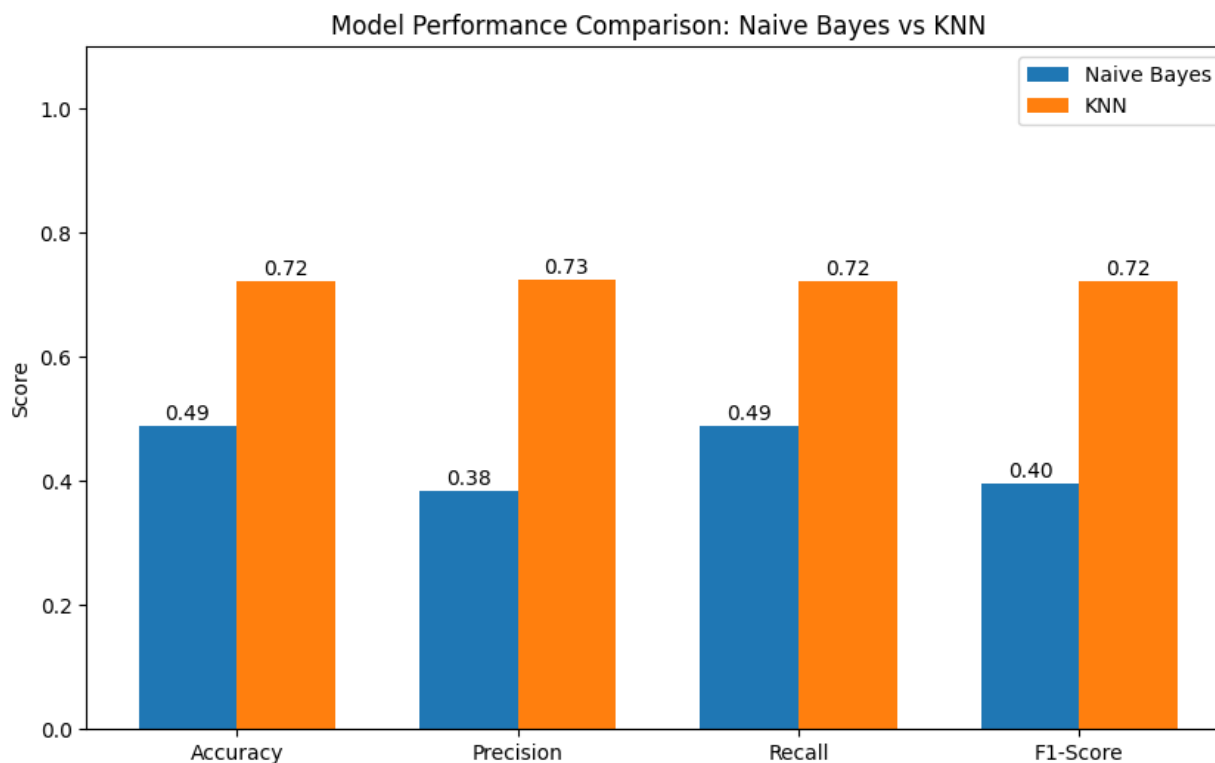


Figure 20 (Model Performance Comparison: Naive Bayes vs KNN)

The comparison clearly demonstrates that the K-Nearest Neighbours model achieved consistently higher performance across all evaluation metrics when compared to the Gaussian Naive Bayes model. This indicates that KNN was more effective at capturing the underlying relationships within the smartphone dataset and produced more reliable classifications across the High, Medium, and Low rating categories. Consequently, KNN was selected as the optimal classification approach for the final smartphone rating category prediction system.

5. Conclusions

5.1 Summary of findings

5.1.1 Regression

The central goal was to predict smartphone price (₹) from hardware specifications. Five models were evaluated using 5-fold cross-validation on log-transformed prices.

Random Forest Regression was the clear winner, achieving a test R^2 of 0.83 outperforming every linear approach. Its train R^2 of 0.9643 indicates some overfitting, but its generalization was still far superior to alternatives.

Linear and Ridge Regression plateaued together at a test R^2 of 0.68. Ridge's best alpha (0.01) produced almost no improvement over Linear Regression, which tells us the features were not suffering from multicollinearity or coefficient inflation.

Polynomial Regression (Degree 2) was catastrophically unstable. While it appeared to learn well on training data ($R^2 = 0.94$), its test R^2 collapsed to -215 on Fold 1 and -23,061,964 on Fold 2, showing extreme overfitting.

5.1.2 Classification

The classification stage of the project focused on two separate prediction tasks:

For price category classification, smartphones were grouped into three categories:

Budget (below ₹15,000), Mid-Range (₹15,000–₹60,000), and Premium (above ₹60,000).

Three classification models were evaluated for this task.

Gaussian Naive Bayes achieved an accuracy of nb%, while **Logistic Regression** produced a similar accuracy of 76%, indicating that both models were limited in capturing the complex relationships between smartphone specifications and pricing categories. In contrast, **Random Forest** achieved the highest classification accuracy of 86%, significantly outperforming the simpler linear and probabilistic approaches. Consequently, Random Forest was selected as the optimal model for smartphone price category classification.

For ratings category classification, smartphones were grouped into three categories:

High (greater than 83), Medium (greater than 79), and Low (79 or below).

Initially, **Gaussian Naive Bayes** achieved an accuracy of 49%, which was considered insufficient for reliable classification. The model's poor performance was primarily due to its assumption that the underlying data follows a Gaussian distribution, which did not align with the skewed distribution observed in the smartphone dataset. To address this

limitation, **K-Nearest Neighbours** was introduced. After evaluating multiple values of K using 5-fold cross-validation, the optimal value was determined to be $K = 3$. The final KNN model achieved an accuracy of 72%, substantially outperforming the Naive Bayes classifier. This improvement demonstrated that similarity-based classification was more suitable for the smartphone rating dataset, making KNN the final selected model for rating category prediction.

5.2 Recommendations

After evaluating multiple machine learning models across different prediction tasks, the following models were identified as the most effective choices for the smartphone analysis pipeline.

For categorising smartphones into Budget, Mid, and Premium brackets, the Random Forest Classifier (RFC) is the strongest choice. It achieved an accuracy of 86.32%, clearly outperforming the Logistic Regression baseline (76.05%) & Naive Bayes (78.42%). This model is recommended for market tiering and inventory classification because its ensemble structure handles complex & overlapping hardware specifications that often confuse simpler linear or probabilistic models.

For predicting user satisfaction through Rating Categories (Poor, Good, and Excellent), the K-Nearest Neighbours (KNN) model is recommended over Naive Bayes. KNN showed a better ability to group devices based on feature similarity, which more closely matches how users judge value and performance across similar smartphones. This makes it suitable for predicting how a new device may be received based on its similarity to existing highly rated or poorly rated devices.

Finally, for predicting exact numerical price points, the Random Forest Regressor is the best-performing model compared to Linear, Ridge, & Polynomial regression models. The regression results show that Random Forest produced the lowest error rates and the highest R-squared score. While Polynomial and Ridge regression slightly improved on basic Linear Regression, they still struggled to model the non-linear relationship between premium features and sharp price increases. This regressor is recommended for accurate Price-to-Specs evaluations to determine whether a smartphone is fairly priced relative to its hardware.

5.3 Limitations

- Ten columns were removed due to excessive missing values, including fast_charge_w, storage_gb, chipset, and display_type. These are meaningful specifications that consumers weigh when evaluating price.
- A measurable portion of a device's price reflects factors entirely absent from our dataset — brand equity, ecosystem lock-in, and material quality. These premiums are unquantifiable from specifications alone, and no amount of tuning can compensate for features the model has never seen. As a result, the model undervalues luxury devices, where the gap between hardware value and actual market price is at its widest.

5.4 Future Work

Option 1:

Future iterations of this project should expand the model comparison to include Gradient Boosting frameworks such as XGBoost and LightGBM. Unlike Random Forest, which averages independent trees, these boosting methods build trees sequentially to correct residual errors, making them highly effective for generalizing complex tabular data. Given the pronounced non-linearity observed in the smartphone market, XGBoost and LightGBM are strong candidates for pushing the predictive model's test R^2 beyond the current 0.83 threshold.

Additionally, subsequent analyses must prioritize advanced feature engineering. Incorporating temporal dynamics, such as device release years, will allow the model to account for technology depreciation, while integrating brand identifiers will help capture the unquantifiable "prestige tax" of premium devices. Finally, broadening the dataset to include global pricing across multiple currencies and deploying the finalized model through a live web interface would elevate this study from a static analysis into a highly robust, real-time pricing tool.

Option 2:

Future work should prioritize Gradient Boosting algorithms like XGBoost and LightGBM. By building trees sequentially to correct residual errors, these models are better equipped to handle non-linear tabular data and could push the current test R^2 beyond 0.83. Furthermore, future datasets must incorporate brand identifiers and temporal features (e.g., release year) to accurately capture luxury brand prestige and real-world tech depreciation. Ultimately, scaling the data to include multi-currency global pricing and

deploying the model via a web interface would successfully transform this research into a robust, real-time market tool.

Option 3:

Advanced Ensemble Modeling: Transitioning from Random Forest to Gradient Boosting frameworks, specifically XGBoost and LightGBM. Because these models build trees sequentially to correct residual errors—rather than averaging independent trees—they are exceptionally well-suited for non-linear tabular data and hold strong potential to push the test R^2 beyond the current 0.83 benchmark.

Expanded Feature Engineering: Future iterations must incorporate qualitative and temporal features. Integrating specific brand data will help quantify the "luxury tax" and ecosystem premium, while tracking 'release year' will allow the model to accurately account for real-world technology depreciation.

Global Scalability & Deployment: Expanding the dataset to encompass global pricing across multiple currencies would significantly increase the model's market applicability. Ultimately, deploying these advanced boosting frameworks via an interactive web interface would transform this static analysis into a robust, real-time predictive tool for consumers and industry analysts.

6. Acknowledgements

Team Contributions

Mohamed Ezzat	Mariam Hassan	Maryam Rageh	Abdelrahman El-Khashab	Abdullah Helmy
Data Sourcing & Cleaning	Logistic Regression	KNN Classification	Linear Regression	Naïve Bayes Classification
Poly-Ridge Regression	Random Forest Classification	Interactive Dashboard	Polynomial Regression	Video
Random Forest Regression	Naïve Bayes Classification	Naïve Bayes Classification	Ridge Regression	
Regression Visualization			Presentation	

Table 5 (Contribution Table)

7. References

Rezayan, Ahad, Reza Poorabbas, and Ali Mansouri. "Applying Neural Network Modeling to Mobile Pricing: A New Approach to Smartphone Price Prediction." *SSRN*, 2025, ssrn.com/abstract=5419155.

"Prediction of Mobile Phone Prices Using Machine Learning." *ACM Digital Library*, ICMLT, May 2024, dl.acm.org/doi/fullHtml/10.1145/3674029.3674031.

Gondaliya, Dev. *Smartphone Specifications Dataset*. Kaggle, 2026, www.kaggle.com/datasets/devgondaliya007/smartphone-specifications-dataset.

Github Repo

<https://github.com/WhipShip/Smartphone-Price-Prediction>